



Definition and Components

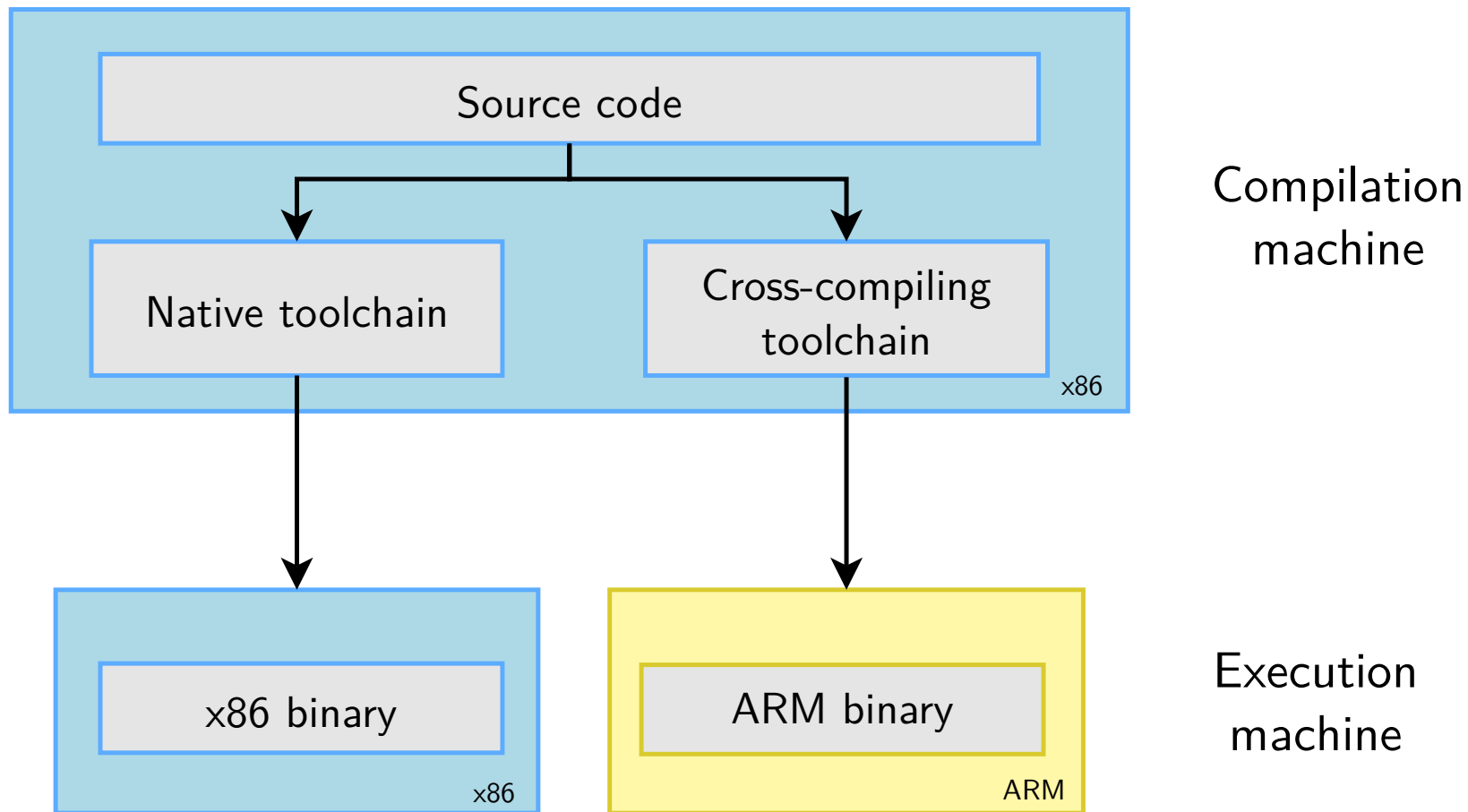


Toolchain definition (1)

- ▶ The usual development tools available on a GNU/Linux workstation is a **native toolchain**
- ▶ This toolchain runs on your workstation and generates binary code for your workstation, usually x86
- ▶ For embedded system development, it is usually impossible or not interesting to use a native toolchain
 - The target is too restricted in terms of storage and/or memory
 - The target is very slow compared to your workstation
 - You may not want to install all development tools on your target.
- ▶ Therefore, **cross-compiling toolchains** are generally used. They run on your workstation but generate code for your target.



Toolchain definition (2)



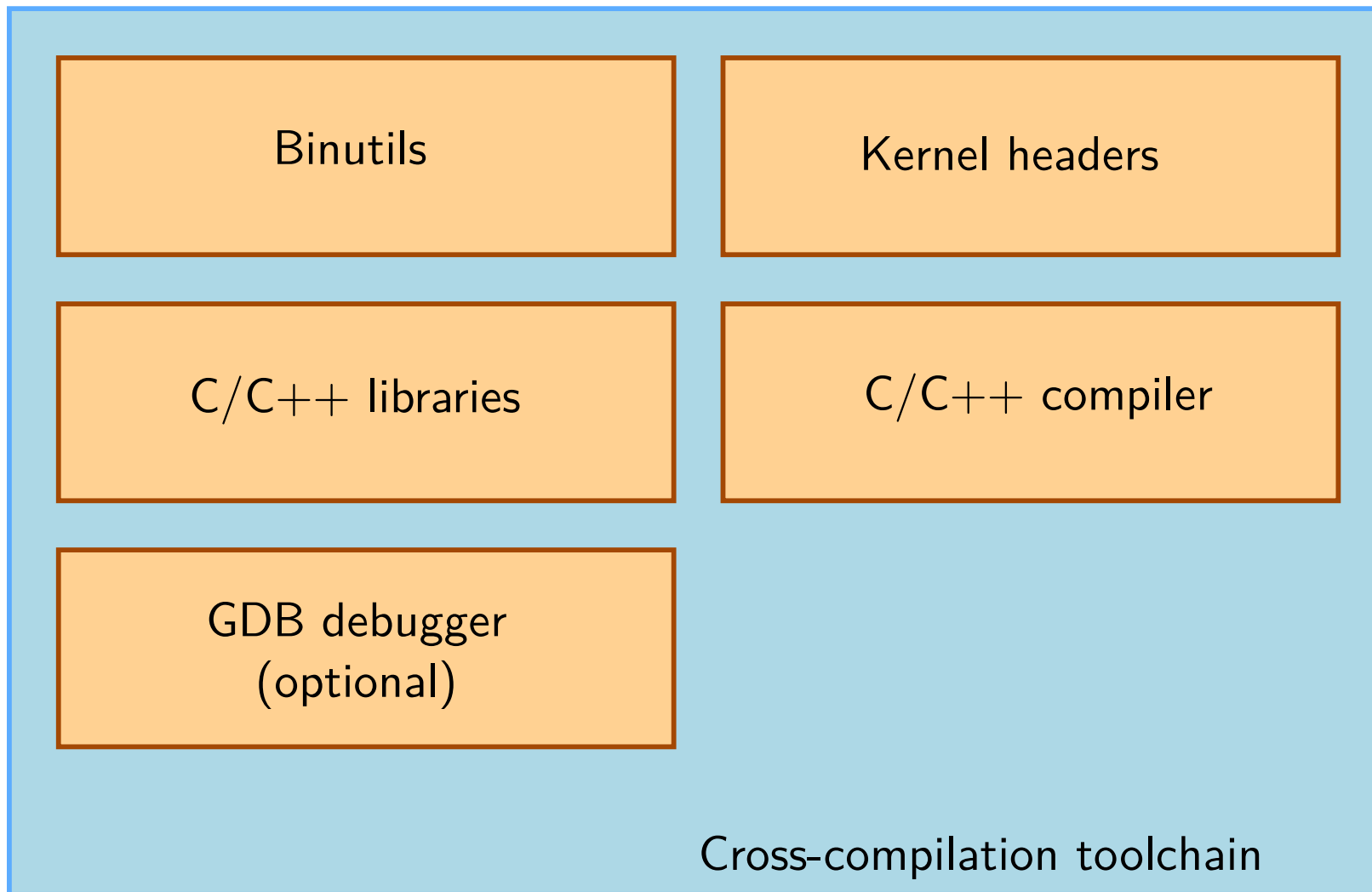


Architecture tuple and toolchain prefix

- ▶ Many UNIX/Linux build mechanisms rely on *architecture tuple* names to identify machines.
- ▶ Examples: `arm-linux-gnueabihf`, `mips64el-linux-gnu`, `arm-vendor-none-eabihf`
- ▶ These tuples are 3 or 4 parts:
 1. The architecture name: `arm`, `riscv`, `mips64el`, etc.
 2. Optionally, a vendor name, which is a free-form string
 3. An operating system name, or `none` when not targeting an operating system
 4. The ABI/C library (see later)
- ▶ This tuple is used to:
 - configure/build software for a given platform
 - as a prefix of cross-compilation tools, to differentiate them from the native toolchain
 - `gcc` $\rightarrow$ `r$_native compiler`
 - `arm-linux-gnueabihf-gcc` $\rightarrow$ `r$_cross-compiler`



Components of gcc toolchains





- ▶ **Binutils** is a set of tools to generate and manipulate binaries (usually with the ELF format) for a given CPU architecture
 - `as`, the assembler, that generates binary code from assembler source code
 - `ld`, the linker
 - `ar`, `ranlib`, to generate `.a` archives (static libraries)
 - `objdump`, `readelf`, `size`, `nm`, `strings`, to inspect binaries. Very useful analysis tools!
 - `objcopy`, to modify binaries
 - `strip`, to strip parts of binaries that are just needed for debugging (reducing their size).
- ▶ GNU Binutils: <https://www.gnu.org/software/binutils/>, GPL license



C/C++ compiler

- ▶ GCC: GNU Compiler Collection, the famous free software compiler
- ▶ <https://gcc.gnu.org/>
- ▶ Can compile C, C++, Ada, Fortran, Java, Objective-C, Objective-C++, Go, etc. Can generate code for a large number of CPU architectures, including x86, ARM, RISC-V, and many others.
- ▶ Available under the GPL license, libraries under the GPL with linking exception.



C/C++ compiler





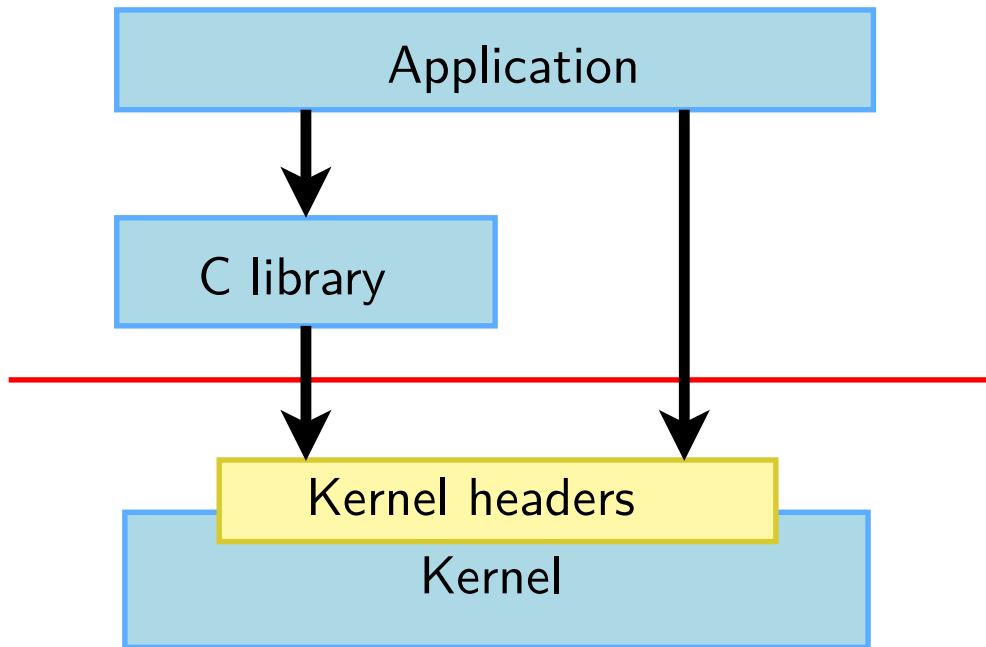
Kernel headers (1)

- ▶ The C standard library and compiled programs need to interact with the kernel
 - Available system calls and their numbers
 - Constant definitions
 - Data structures, etc.
- ▶ Therefore, compiling the C standard library requires kernel headers, and many applications also require them.
- ▶ Available in `<linux/...>` and `<asm/...>` and a few other directories corresponding to the ones visible in `include/uapi/` and in `arch/<arch>/include/uapi` in the kernel sources



Kernel headers (1)

- ▶ The kernel headers are extracted from the kernel sources using the `headers_install` kernel Makefile target.





Kernel headers (2)

- ▶ System call numbers, in `<asm/unistd.h>`

```
#define __NR_exit      1
#define __NR_fork      2
#define __NR_read      3
```

- ▶ Constant definitions, here in `<asm-generic/fcntl.h>`, included from `<asm/fcntl.h>`, included from `<linux/fcntl.h>`

```
#define O_RDWR 00000002
```

- ▶ Data structures, here in `<asm/stat.h>` (used by the `stat` command)

```
struct stat {
    unsigned long st_dev;
    unsigned long st_ino;
    [...]
};
```



Kernel headers (3) The kernel to user space interface is

backward compatible

- ▶ Kernel developers are doing their best to **never** break existing programs when the kernel is upgraded. Otherwise, users would stick to older kernels, which would be bad for everyone.
- ▶ Hence, binaries generated with a toolchain using kernel headers older than the running kernel will work without problem, but won't be able to use the new system calls, data structures, etc.
 - Binaries generated with a toolchain using kernel headers newer than the running kernel might work only if they don't use the recent features, otherwise they will break.

What to remember: updating your kernel shouldn't break your programs;
it's usually fine to keep an old toolchain as long as it works fine for your project.



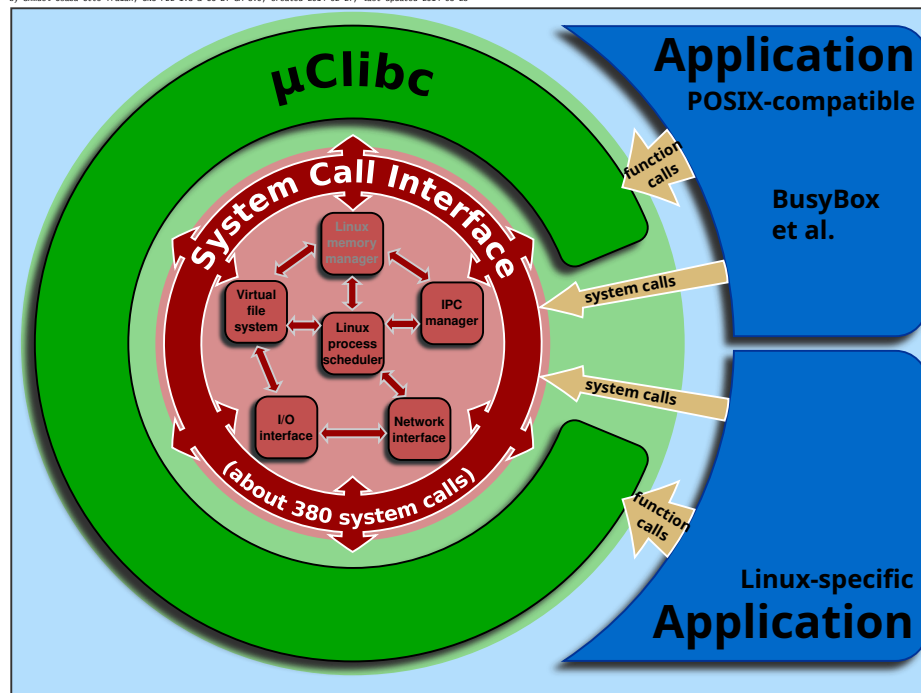
C standard library

- ▶ The C standard library is an essential component of a Linux system.
 - Interface between the applications and the kernel
 - Provides the well-known standard C API to ease application development
- ▶ Several C standard libraries are available: *glibc*, *uClibc*, *musl*, *klibc*, *newlib*...
- ▶ The choice of the C standard library must be made at cross-compiling toolchain generation time, as the GCC compiler is compiled against a specific C standard library.



C standard library

by Shmuel Csaba Otto Traian; GNU FDL 1.3 & CC-BY-SA 3.0; created 2014-02-27, last updated 2014-03-25



Source: Wikipedia (<https://bit.ly/2zrGve2>)

- ▶ License: LGPL
- ▶ C standard library from the GNU project
- ▶ Designed for performance, standards compliance and portability
- ▶ Found on all GNU / Linux host systems
- ▶ Of course, actively maintained
- ▶ By default, quite big for small embedded systems. On armv7hf, version 2.31: `libc`: 1.5 MB, `libm`: 432 KB, source: <https://toolchains.bootlin.com>
- ▶ <https://www.gnu.org/software/libc/>



Image source



- ▶ <https://uclibc-ng.org/>
- ▶ A continuation of the old uClibc project, license: LGPL
- ▶ Lightweight C standard library for small embedded systems
 - High configurability: many features can be enabled or disabled through a menuconfig interface.
 - Supports most embedded architectures, including MMU-less ones (ARM Cortex-M, Blackfin, etc.). The only standard library supporting ARM noMMU.
 - No guaranteed binary compatibility. May need to recompile applications when the library configuration changes.
 - Some features may be implemented later than on glibc (real-time, floating-point operations...)
 - Focus on size (RAM and storage) rather than performance
 - Size on armv7hf, version 1.0.34: `libc`: 712 KB, source: <https://toolchains.bootlin.com>
- ▶ Actively supported, supported by Buildroot but not by Yocto Project.



musl C standard library

<https://www.musl-libc.org/>

- ▶ A lightweight, fast and simple standard library for embedded systems
 - ▶ Created while uClibc's development was stalled
 - ▶ In particular, great at making small static executables, which can run anywhere, even on a system built from another C standard library.
 - ▶ More permissive license (MIT), making it easier to release static executables.
- We will talk about the requirements of the LGPL license (glibc, uClibc) later.
- ▶ Supported by build systems such as Buildroot and Yocto Project.

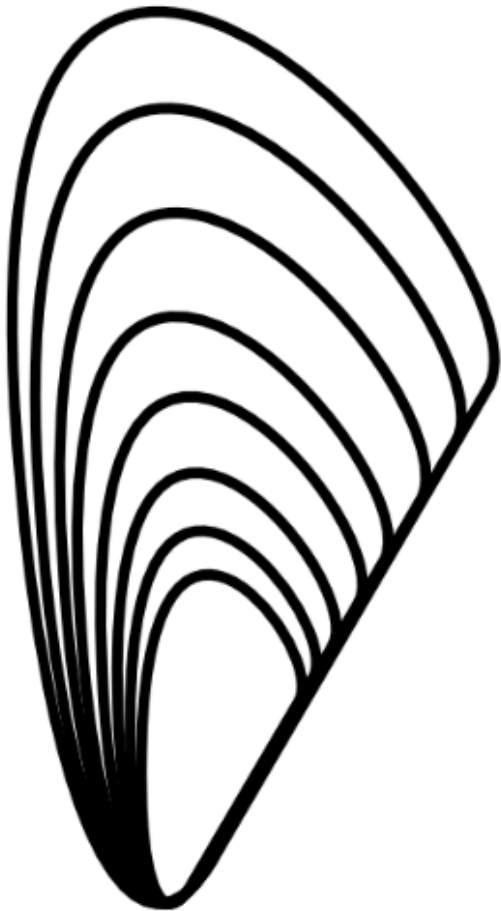


musl C standard library

- ▶ Used by the Alpine Linux lightweight distribution (<https://www.alpinelinux.org/>)
- ▶ Size on armv7hf, version 1.2.0: `libc`: 748 KB, source: <https://toolchains.bootlin.com>



musl C standard library





Other smaller C libraries

- ▶ Several other smaller C libraries exist, but they do not implement the full POSIX interface required by most Linux applications
- ▶ They can run only relatively simple programs, typically to make very small static executables and run in very small root filesystems.
- ▶ Therefore not commonly used in most embedded Linux systems
- ▶ Choices:
 - Newlib, <https://sourceware.org/newlib/>, maintained by Red Hat, used mostly in Cygwin, in bare metal and in small POSIX RTOS.
 - Klibc, <https://en.wikipedia.org/wiki/Klibc>, from the kernel community, designed to implement small executables for use in an *initramfs* at boot time.



Advice for choosing the C standard library

- ▶ Advice to start developing and debugging your applications with *glibc*, which is the most standard solution
- ▶ If you have size constraints, try to compile your app and then the entire filesystem with *uClibc* or *musl*
 - The size advantage of *uClibc* or *musl*, which used to be a significant argument, is less relevant with today's storage capacities.
 - Smaller binaries and filesystems remain useful when optimizing boot time, though, typically booting on a filesystem loaded in RAM, and to reduce the size of container and virtual machine images (one of the use cases of Alpine Linux).
- ▶ If you run into trouble, it could be because of missing features in the C standard library.
- ▶ In case you wish to make static executables, *musl* will be an easier choice in terms of licensing constraints.



Linux vs. bare-metal toolchain

▶ A **Linux toolchain**

- is a toolchain that includes a Linux-ready C standard library, which uses the Linux system calls to implement system services
- can be used to build Linux user-space applications, but also bare-metal code (firmware, bootloader, Linux kernel)
- is identified by the `linux` OS identifier in the toolchain tuple: `arm-linux`, `arm-none-linux-gnueabi`

▶ A **bare metal toolchain**

- is a toolchain that does not include a C standard library, or a very minimal one that isn't tied to a particular operating system
- can be used to build only bare-metal code (firmware, bootloader, Linux kernel)
- is identified by the `none` OS identifier in the toolchain tuple: `arm-none-eabi`, `arm-none-none-eabi` (vendor is `none`, OS is `none`)



An alternate compiler suite: LLVM

- ▶ Most Embedded Linux projects use toolchains based on the GNU project: GCC compiler, binutils, GDB debugger
- ▶ The LLVM project has been developing an alternative compiler suite:
 - Clang, C/C++ compiler, <https://clang.llvm.org/>
 - LLDB, debugger, <https://lldb.llvm.org/>
 - LLD, linker, <https://lld.llvm.org/>
 - and more, see <https://llvm.org/>
- ▶ While they are used by several high-profile projects, they are not yet in widespread use in most Embedded Linux projects.
- ▶ Initially had better code optimization and diagnostics than GCC, but thanks to having competition, GCC has improved significantly in this area.
- ▶ Available under MIT/BSD licenses
- ▶ <https://en.wikipedia.org/wiki/LLVM>